LAPORAN JOBSHEET
“15 - Implementasi Sistem Registrasi (Database Integration)”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

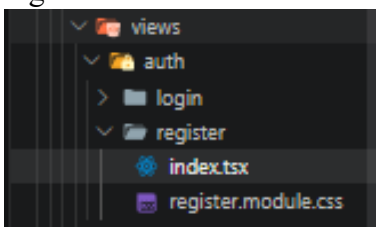
POLITEKNIK NEGERI MALANG

2026

LANGKAH PRAKTIKUM

Bagian 1 – Membuat Register View

1. Buat folder pada views dengan nama register dan tambahkan 2 file yaitu index.tsx dan register.module.scss



2. Modifikasi file index.tsx (pada folder views/auth/register/index.tsx)

```
index.tsx ×   register.tsx
P2 > Praktikum > next-routing > src > views > auth > register > index.tsx > ...
1   import Link from "next/link";
2   import styles from "./register.module.css";
3
4   const RegisterPage = () => {
5     return (
6       <div className={styles.register}>
7         <h1 className={styles.register_title}>Register</h1>
8         <Link href="auth/login">Already have an account? Login here</Link>
9       </div>
10    );
11  };
12
13  export default RegisterPage;
```

3. Buka file register.tsx pada folder pages/auth/register.tsx
4. Modifikasi file register.tsx (pada folder pages/auth/register.tsx)

```
P2 > Praktikum > next-routing > src > pages > auth > register.tsx > HalamanRegister
1   import RegisterPage from "@/views/auth/register";
2
3   const HalamanRegister = () => {
4     return <RegisterPage />;
5   };
6
7   export default HalamanRegister;
8
```

5. Modifikasi register.module.scss

```
P2 > Praktikum > next-routing > src > views > auth > register > register.module.css > .register_title
1   .register {
2     padding: 40px;
3   }
4
5   .register_title {
6     font-size: 28px;
7     font-weight: bold;
8     margin-bottom: 20px;
9   }
```

6. Tambahkan form inputan pada file index.tsx (pada folder views/auth/register/index.tsx) Form berisi: Email, Full name, Password, dan Button

register

P2 > Praktikum > next-routing > src > views > auth > register > index.tsx > ...

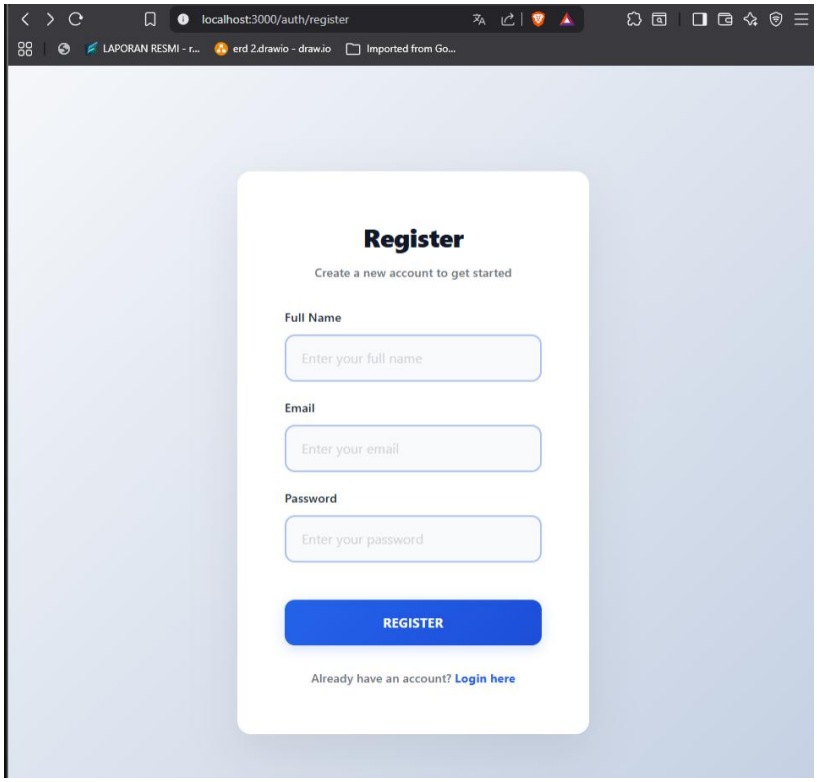
```
7   const RegisterPage = () => {
27
28    return (
29      <div className={styles.register}>
30        <h1 className={styles.register_title}>Register</h1>
31        <form className={styles.register_form} onSubmit={handleSubmit}>
32          <div className={styles.form_group}>
33            <label htmlFor="fullName">Full Name</label>
34            <input
35              type="text"
36              id="fullName"
37              name="fullName"
38              value={formData.fullName}
39              onChange={handleChange}
40              placeholder="Enter your full name"
41              required
42            />
43          </div>
44
45          <div className={styles.form_group}>
46            <label htmlFor="email">Email</label>
47            <input
48              type="email"
49              id="email"
50              name="email"
51              value={formData.email}
52              onChange={handleChange}
53              placeholder="Enter your email"
54              required
55            />
56          </div>
57
58          <div className={styles.form_group}>
59            <label htmlFor="password">Password</label>
60            <input
61              type="password"
62              id="password"
63              name="password"
64              value={formData.password}
65              onChange={handleChange}
66              placeholder="Enter your password"
67              required
68            />
69          </div>
70
71          <button type="submit" className={styles.register_button}>
72            Register
73          </button>
74        </form>
75
76        <p className={styles.login_link}>
77          Already have an account? <Link href="/auth/login">Login here</Link>
```

7. Modifikasi register.module.scss

P2 > Praktikum > next-routing > src > views > auth > register > register.module.scss > .register

```scss
.form_group {
  flex-direction: column;
  gap: 0.5rem;

  label {
    color: #1f2937;
    font-weight: 600;
    font-size: 0.95rem;
    display: block;
    margin: 0;
  }

  input {
    padding: 0.75rem 1rem;
    border: 1px solid #e5e7eb;
    border-radius: 0.5rem;
    font-size: 1rem;
    transition: all 0.2s;
    font-family: inherit;
    width: 100%;
    box-sizing: border-box;

    &:focus {
      outline: none;
      border-color: #2563eb;
      box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.1);
    }

    &::placeholder {
      color: #9ca3af;
    }

    &:invalid {
      border-color: #ef4444;
    }
  }
}

.register_button {
  background-color: #2563eb;
  color: white;
  font-weight: 600;
  padding: 0.75rem 1.5rem;
  border-radius: 0.5rem;
  transition: all 0.2s;
  border: none;
  cursor: pointer;
  font-size: 1rem;
  margin-top: 1rem;
  width: 100%;

  &:hover {
    background-color: #1d4ed8;
```

8. Jalankan browsernya http://localhost:3000/auth/register sehingga tampilan sebagai berikut

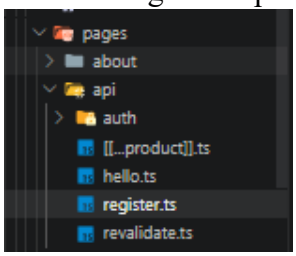


Bagian 2 – Membuat API Register

1. Buka file servicefirebase.ts pada folder src/utils/db dan modifikasi

```
P2 > Praktikum > next-routing > src > utils > db > servicesfirebase.ts > signUp
46  export async function signUp(
47    userData: {
48      email: string;
49      password: string;
50      fullName: string;
51    },
52    callback: (response: {
53      status: "success" | "error";
54      message: string;
55      userId?: string;
56      error?: unknown;
57    }) => void,
58  ) {
59    try {
60      // Check if user with this email already exists
61      const q = query(
62        collection(db, "users"),
63        where("email", "==", userData.email),
64      );
65      const querySnapshot = await getDocs(q);
66      const existingUsers = querySnapshot.docs.map((doc) => ({
67        id: doc.id,
68        ...doc.data(),
69      }));
70
71      // If user already exists, return error
72      if (existingUsers.length > 0) {
73        callback({
74          status: "error",
75          message: "User with this email already exists",
76        });
77        return;
78      }
79
80      // If user doesn't exist, create new user
81      const newUser = await addDoc(collection(db, "users"), {
82        email: userData.email,
83        password: userData.password, // Note: Consider hashing this in production
84        fullName: userData.fullName,
85        createdAt: Timestamp.now(),
86      });
87
88      callback({
89        status: "success",
90        message: "User registered successfully",
91        userId: newUser.id,
92      });
93    } catch (error) {
94      console.error("Error during sign up:", error);
95      callback({
96        status: "error",
97        message: "An error occurred during registration. Please try again.",
```

2. Buat file register.ts pada folder api



3. Modifikasi file register.ts

P2 > Praktikum > next-routing > src > pages > api > register.ts > handler

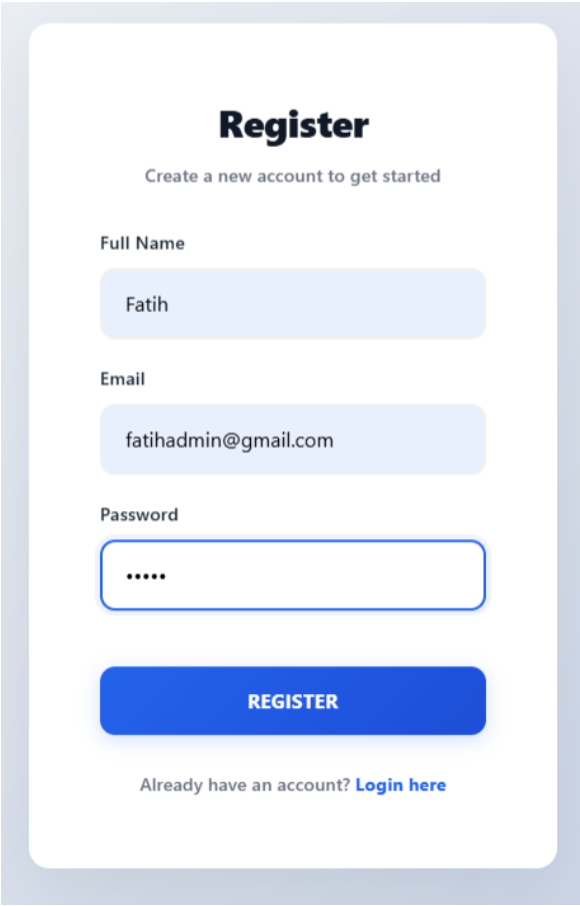
```python
import { signUp } from "@/utils/db/servicesfirebase";
import type { NextApiRequest, NextApiResponse } from "next";

type Data = {
  name: string;
  alamat: string;
};

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>
) {
  if (req.method === "POST") {
    await signUp(req.body, (result: { status: string; message: string }) => {
      if (result.status === "success") {
        res.status(200).json({ name: result.message, alamat: "" });
      } else {
        res.status(400).json({ name: result.message, alamat: "" });
      }
    });
  } else {
    res.status(405).json({ name: "Method not allowed", alamat: "" });
  }
}
```

4. Modifikasi index.tsx pada folder register (tambahkan beberapa code)

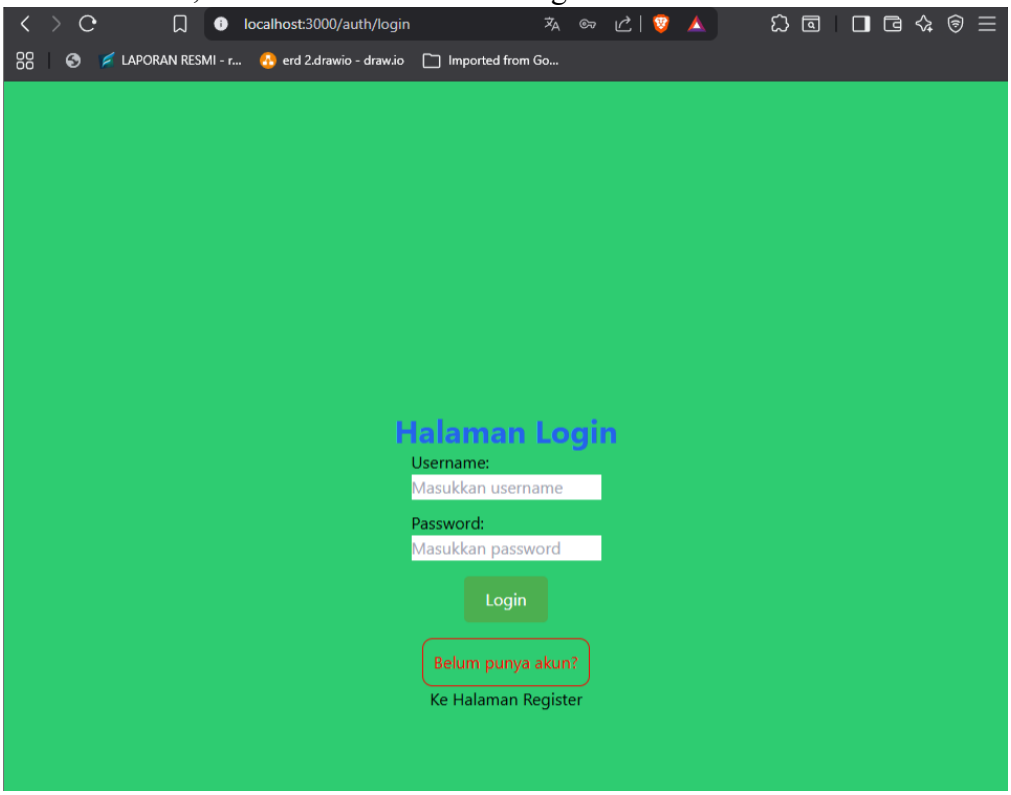
P2 > Praktikum > next-routing > src > views > auth > register > index.tsx > RegisterPage

```python
"use client";

import Link from "next/link";
import { useState } from "react";
import styles from "./register.module.scss";
import { useRouter } from "next/router";

const RegisterPage = () => {
  const [isLoading, setIsLoading] = useState(false);
  const { push } = useRouter();
  const [error, setError] = useState("");

  const [formData, setFormData] = useState({
    fullName: "",
    email: "",
    password: "",
  });

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value,
    }));
  };

  const handleSubmit = async(e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    const form = e.currentTarget;
    const formData = new FormData(form);
    const email = formData.get("email") as string;
    const password = formData.get("password") as string;
    const fullName = formData.get("fullName") as string;
    const response = await fetch("/api/register", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ email, password, fullName }),
    });

    if (response.status === 200) {
      form.reset();
      setIsLoading(false);
      push("/auth/login");
    } else {
      setIsLoading(false);
      setError(response.status === 400 ? "User already exists" : "Failed to regist
    }
    console.log("Form submitted:", formData);
    // Add your registration logic here
```

5. Buka browser http://localhost:3000/auth/register isikan data dan klik register. Jika berhasil maka akan masuk ke menu login



Setelah submit, maka akan diarahkan ke login



Bagian 3 – Install bcrypt

1. npm install bcrypt –force

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework> npm install bcrypt --force
npm warn using --force Recommended protections disabled.

added 3 packages in 5s
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework> npm install bcrypt --force
npm warn using --force Recommended protections disabled.

up to date, audited 4 packages in 1s

found 0 vulnerabilities
```

2. npm install --save-dev @types/bcrypt –force

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework> npm install --save-dev @types/bcrypt --force
npm warn using --force Recommended protections disabled.

added 3 packages, and audited 7 packages in 2s

found 0 vulnerabilities
```

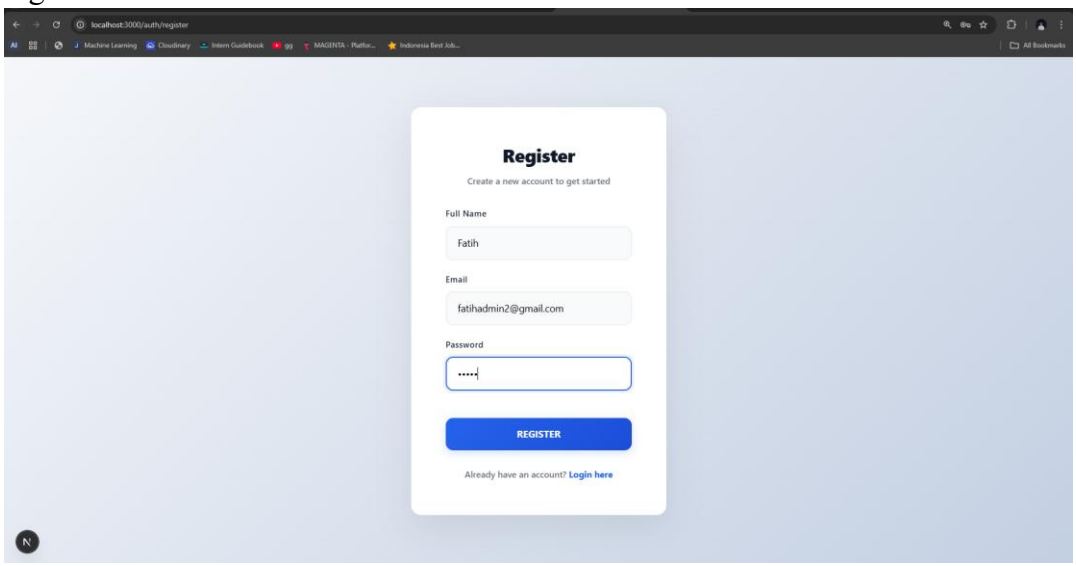
3. Buka file servicefirebase.ts pada folder src/utils/db dan modifikasi

```
  query,
  where,
  addDoc,
  Timestamp,
} from "firebase/firestore";
import app from "./firebase";
import bcrypt from "bcrypt";

const db = getFirestore(app);

export async function retrieveProducts<T>(
  collectionName: string,
```

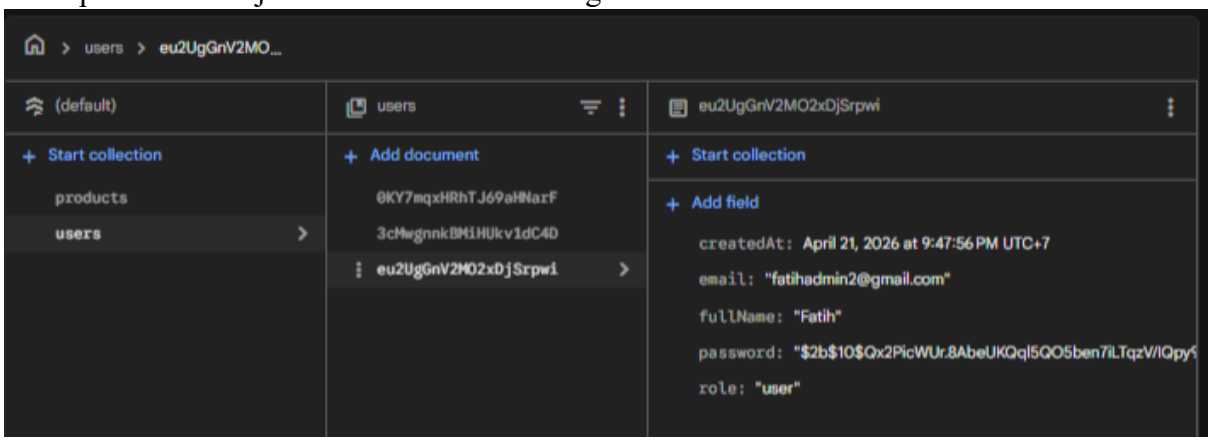
```
export async function signUp(
  userData: {
    email: string;
    password: string;
    fullName: string;
    role?: string;
  },
```

```
  // If user doesn't exist, create new user
  const newUser = await addDoc(collection(db, "users"), {
    email: userData.email,
    password: await bcrypt.hash(userData.password, 10),
    fullName: userData.fullName,
    role: userData.role || "user",
    createdAt: Timestamp.now(),
  });
```

4. Jalankan browser http://localhost:3000/auth/register dan input data setelah itu klik register



5. Buka pada firebase jika berhasil maka data register akan masuk



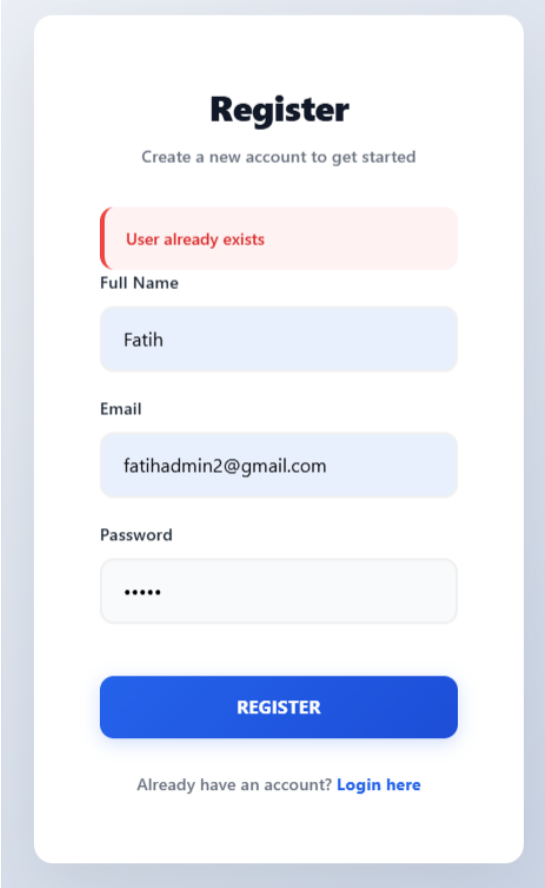
6. Jika user memasukkan data yang sama sistem tidak akan memproses tetapi permasalahannya user memasukkan data yang sama tidak ada pemberitahuan pada layar maka dari itu perlu ada perubahan pada code index.tsx pada folder views/auth/register

```
{error && <div className={styles.error_message}>{error}</div>}
```

7. Modifikasi juga pada register.module.scss

```
.error_message {
  background-color: #fef2f2;
  color: #dc2626;
  padding: 1rem 1.125rem;
  border-radius: 0.75rem;
  font-size: 0.875rem;
  border-left: 4px solid #ef4444;
  font-weight: 500;
  animation: slideIn 0.3s ease;
}
```

8. Jika berhasil maka hasilnya seperti berikut



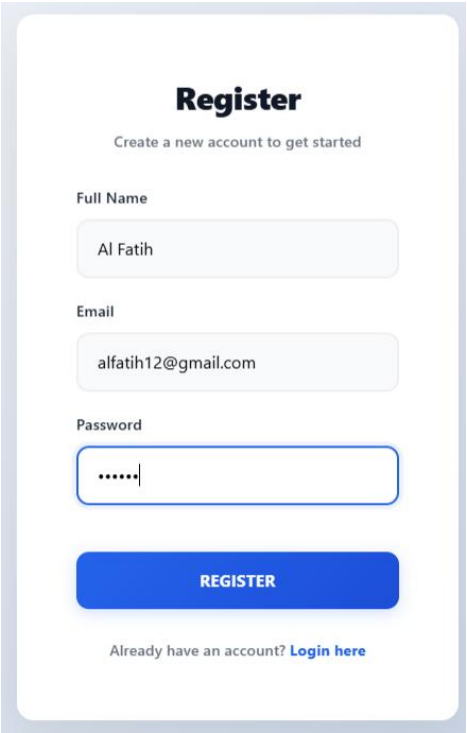
9. Tambakan loading dengan menambahkan kode pada index.tsx

```
const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  setIsLoading(true);
```

Jika berhasil maka hasilnya akan muncul loading saat klik register

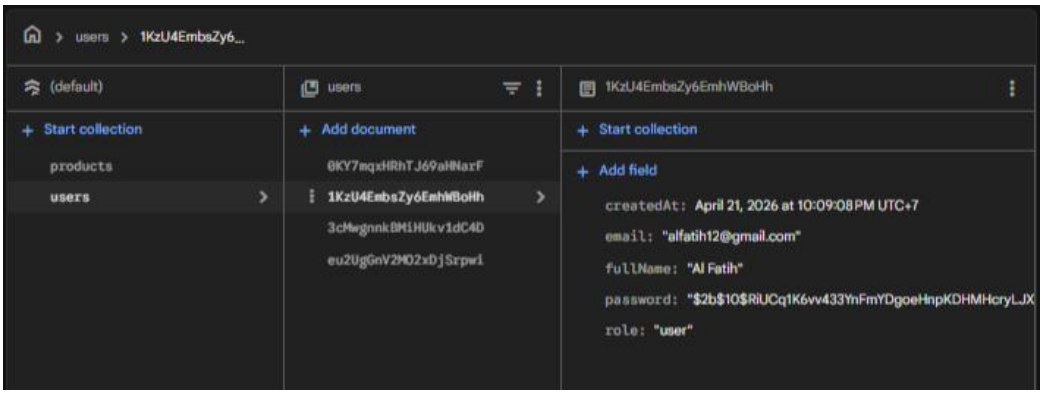
PENGUJIAN

1. Uji 1 – Register Baru
   Input:
   - Email baru

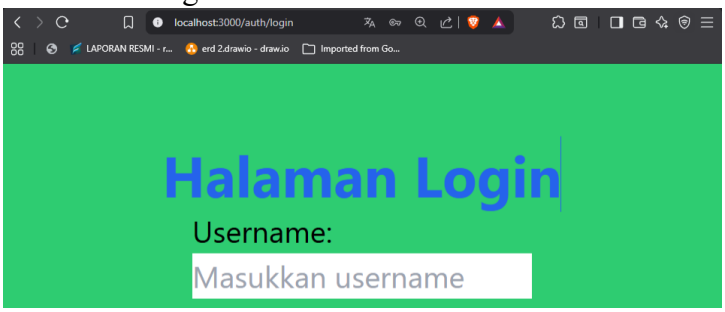


   Hasil:

   - Data tersimpan di Firestore
   - Password ter-hash



   - Redirect ke login



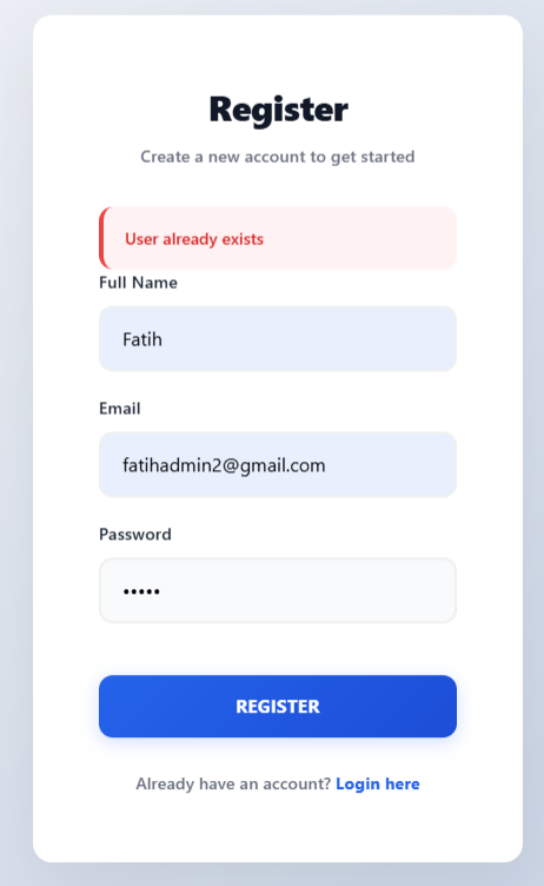
2. Uji 2 – Email Sudah Ada
   Input:
   - Email yang sama

   Hasil:

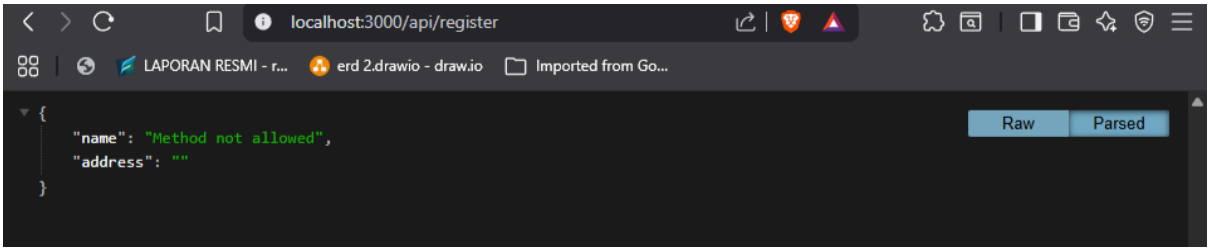
   - Error 400

```
POST /api/register 400 in 340ms (compile: 3ms, render: 337ms)
```

   - Message: Email already exists

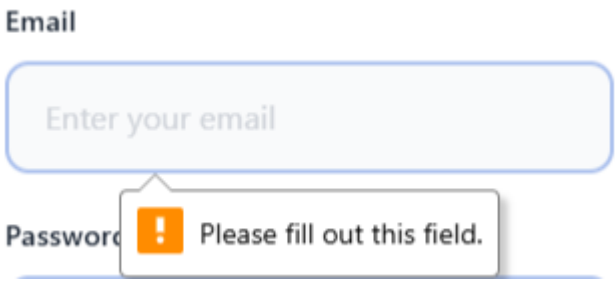


3. Uji 3 – Method GET
   Akses: /api/register
   Hasil:
   - 405 Method Not Allowed



TUGAS PRAKTIKUM

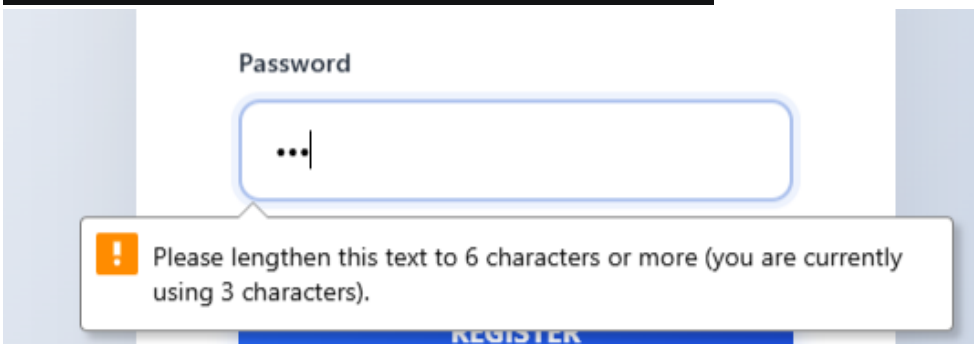
1. Implementasikan register terhubung database. (sudah)
2. Tambahkan validasi:
   - Email wajib



   sudah karena form email required
   - Password minimal 6 karakter

```
if (!password || password.length < 6) {
  setIsLoading(false);
  setError("Password must be at least 6 characters");
  return;
}
```

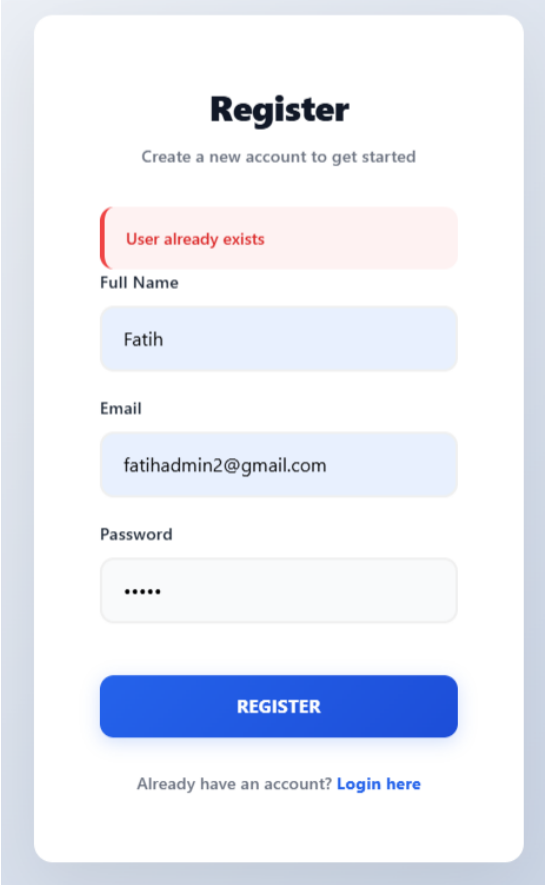
```
<div className={styles.form_group}>
  <label htmlFor="password">Password</label>
  <input
    type="password"
    id="password"
    name="password"
    value={formData.password}
    onChange={handleChange}
    placeholder="Enter your password"
    required
    minLength={6}
  />
</div>
```



3. Tambahkan role default "member".

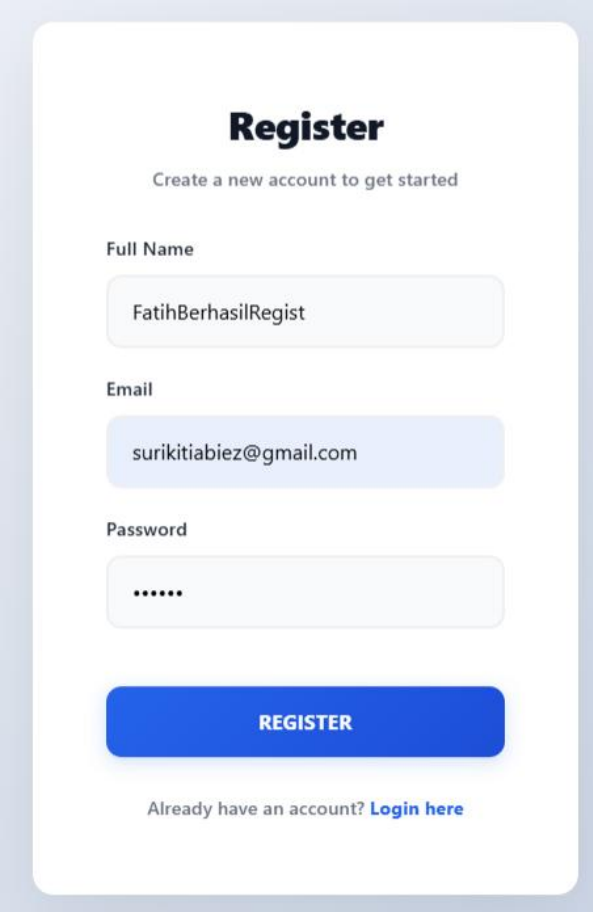
```
// If user doesn't exist, create new user
const newUser = await addDoc(collection(db, "users"), {
  email: userData.email,
  password: await bcrypt.hash(userData.password, 10),
  fullName: userData.fullName,
  role: userData.role || "member",
  createdAt: Timestamp.now(),
});
```

4. Tampilkan pesan error di UI.

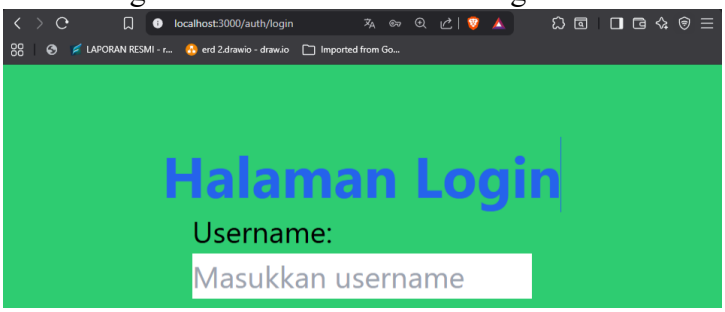


5. Screenshot hasil:

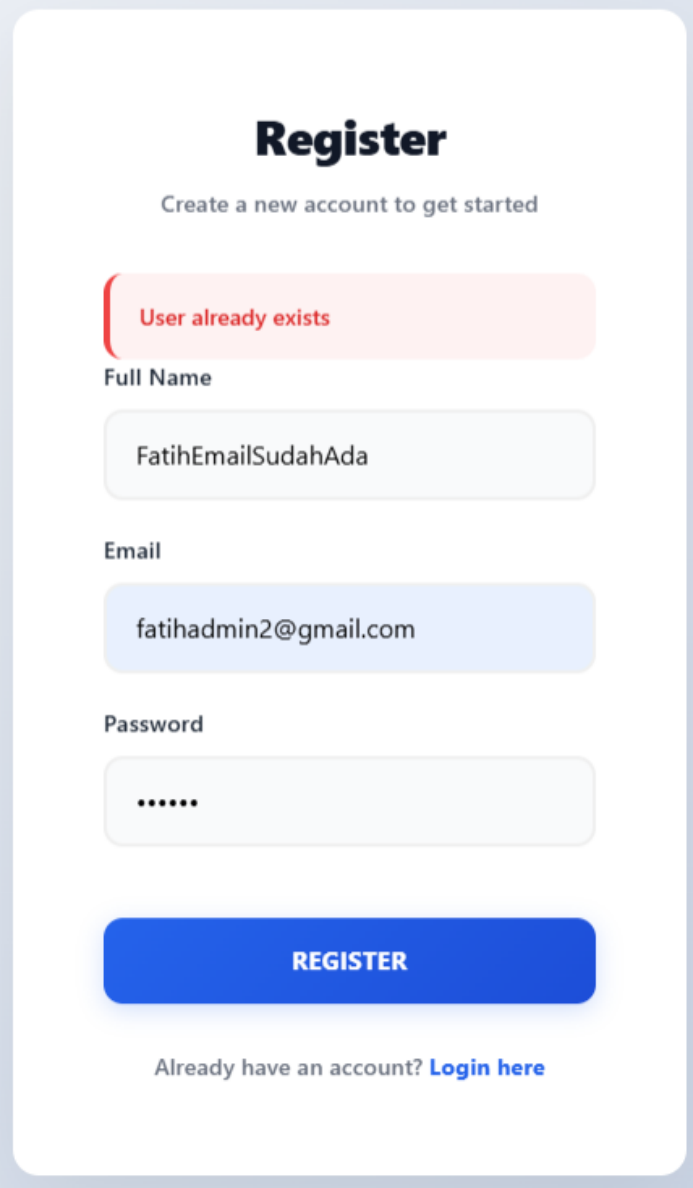
- Register sukses



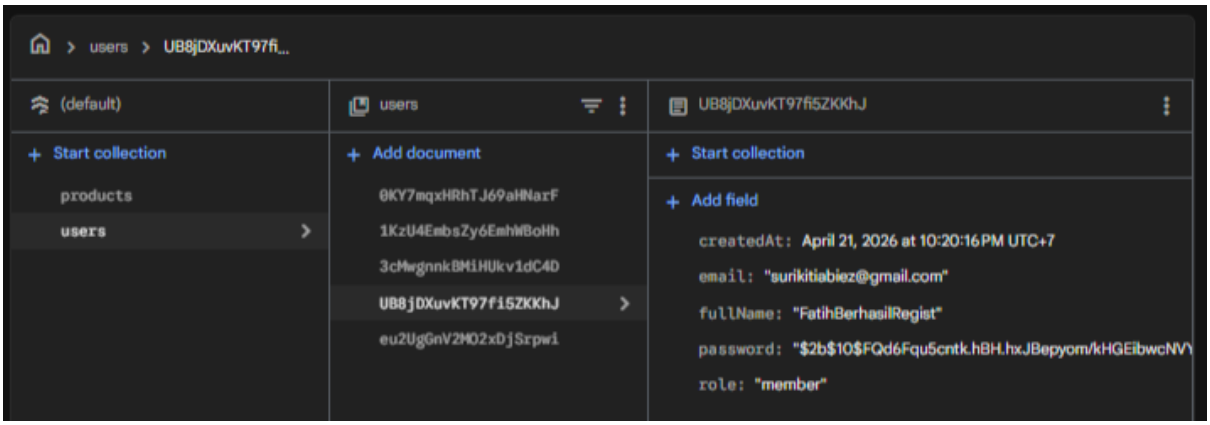
Setelah registrasi auto direct ke /auth/login



- Email sudah ada



- Database Firestore



PERTANYAAN ANALISIS

1. **Mengapa password harus di-hash?**
   Password harus di-hash agar tidak disimpan dalam bentuk asli (plain text), sehingga jika database bocor, attacker tidak langsung mengetahui password pengguna; hashing (ditambah salt) membuat password sulit dibalik ke bentuk awal.

2. **Apa perbedaan addDoc dan setDoc?**
   ddDoc digunakan untuk menambahkan data dengan ID otomatis yang dibuat oleh Firestore, sedangkan setDoc digunakan untuk menyimpan data dengan ID yang kita tentukan sendiri (bisa overwrite jika ID sudah ada).

3. **Mengapa perlu validasi method POST?**
   Validasi method POST diperlukan untuk memastikan endpoint hanya menerima request yang sesuai (misalnya untuk create data), sehingga mencegah penyalahgunaan seperti akses via GET yang tidak semestinya atau potensi celah keamanan

4. **Apa risiko jika email tidak dicek unik?**
   Jika email tidak dicek unik, bisa terjadi duplikasi akun dengan email yang sama, yang berisiko menyebabkan konflik data, masalah login (ambigu), dan potensi penyalahgunaan identitas.

5. **Apa fungsi role pada user?**
   Role pada user berfungsi untuk mengatur hak akses (authorization), misalnya membedakan antara admin dan user biasa, sehingga setiap pengguna hanya bisa mengakses fitur sesuai izin yang diberikan.